

Phase 8: Recursive Merge Sort — Full Structured Beginner Note

1. Current Progress Before Phase 8

Before Phase 8, you have already studied the main foundations of sorting.

Phase 1: Sorting Foundations

You learned the important words used to understand sorting algorithms:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(n)$	Work grows directly with input size
$O(n^2)$	Work grows very quickly, often because of nested loops
$O(n \log n)$	Usually a fast class for comparison-based sorting
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, "Does it sort?"

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

Bubble Sort compares neighboring values and swaps them if they are in the wrong order.

Example:

8 5

Since:

8 > 5

Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
- It uses swaps.
- One full scan is called a pass.
- After each pass, one large value is fixed at the end.
- After k passes, the k largest values are fixed at the end.
- It can be adaptive if we use a flag.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$ with the adaptive flag.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

Phase 3: Insertion Sort

Insertion Sort builds a sorted left part.

Example:

[5 8] 7 3 2

The left part [5 8] is sorted.

Now we insert 7 into the correct position:

[5 7 8] 3 2

Important points from Insertion Sort:

- It inserts one value into a sorted left part.
- It starts from index `1` because the first element alone is already sorted.
- It stores the selected value in `x`.
- It shifts bigger values to the right.
- It is naturally adaptive.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

Phase 4: Selection Sort

Selection Sort selects the smallest value from the unsorted part and puts it at the front.

Example:

8 6 3 2 5 4

The smallest value is `2`, so Selection Sort swaps it with the first value:

2 6 3 8 5 4

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.
- `i` is the position being filled.
- `j` scans the unsorted part.
- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.
- Best-case time is $O(n^2)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- After `k` passes, the `k` smallest values are fixed at the front.

Phase 5: Quick Sort Partitioning

Phase 5 introduced the heart of Quick Sort: partitioning.

Partitioning means:

Choose one value, called the pivot, and place it in its final sorted position.

Example:

50 70 60 90 40 80 10 20 30 ∞

The pivot is 50.

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

Now 50 is in its final sorted position.

But the left side is not fully sorted:

40 30 20 10

And the right side is not fully sorted:

80 90 60 70

So partitioning alone does not fully sort the array.

Phase 6: Quick Sort Recursion and Complexity

In Phase 6, you completed Quick Sort.

Quick Sort works like this:

Partition the array.
Sort the left side recursively.
Sort the right side recursively.

Simple memory trick:

Partition, sort left, sort right.

Important points from Quick Sort:

- Quick Sort uses divide and conquer.
- It chooses a pivot.
- Partitioning places the pivot in its correct position.
- Recursion sorts the smaller parts.
- Best-case time is $O(n \log n)$.
- Average-case time is $O(n \log n)$.
- Worst-case time is $O(n^2)$.
- First-element pivot Quick Sort can be bad for already sorted data.
- It is usually not stable.
- It is mostly in-place for array storage, but recursion uses stack memory.

Phase 7: Merging and Iterative Merge Sort

Phase 7 introduced the most important operation behind Merge Sort:

Merging.

Merging means combining two already sorted lists into one bigger sorted list.

Example:

A = [2, 6, 10]
B = [3, 7]

Both lists are already sorted.

After merging:

C = [2, 3, 6, 7, 10]

Important points from Phase 7:

- Merging works only when the two input parts are already sorted.
- Merging uses `i`, `j`, and `k`.
- `i` reads the left section.
- `j` reads the right section.
- `k` writes into the temporary array.
- A temporary array `B[]` is used to safely store the merged result.
- Leftover values must be copied after one side finishes.
- The merged values must be copied from `B[]` back to `A[]`.
- Iterative Merge Sort uses loops.
- Recursive Merge Sort uses recursion.

Phase 8 now teaches Recursive Merge Sort.

2. Why Phase 8 Comes After Phase 7

Recursive Merge Sort depends completely on the merge operation.

If you do not understand merging, Recursive Merge Sort will feel confusing.

Phase 7 taught this idea:

Two sorted parts can be merged into one bigger sorted part.

Phase 8 adds this idea:

Use recursion to create those sorted parts.

So Phase 8 is really:

Recursive Merge Sort = Recursion + Merge function

3. Main Goal of Phase 8

The main goal of Phase 8 is to understand Recursive Merge Sort.

In simple words:

Recursive Merge Sort breaks an array into smaller parts, then merges those parts back in sorted order.

Phase 8 covers:

- Divide and conquer strategy.
- Top-down recursion.
- The `low`, `mid`, and `high` parameters.
- The recursive `MergeSort()` function.
- The `Merge()` function from the previous phase.
- Base case of recursion.
- Post-order merging.
- Recursion tree.
- Dry run on `[8, 3, 7, 4]`.
- Full C program.
- Time complexity.
- Space complexity.
- Stability.
- Common beginner mistakes.

4. What Is Recursive Merge Sort?

Recursive Merge Sort is a sorting algorithm that uses two main ideas:

1. Divide the array into smaller parts.
2. Merge the sorted parts back together.

It keeps dividing the array until each part has only one element.

Why one element?

Because one element is always sorted.

Example:

[8]

There is nothing to compare and nothing to rearrange.

After the array has been divided into one-element parts, Merge Sort starts merging those parts back together in sorted order.

5. Simple Memory Trick

Remember Recursive Merge Sort like this:

```
Split, split, split.  
Then merge, merge, merge.
```

Another memory trick:

```
Go down by dividing.  
Come up by merging.
```

6. Divide and Conquer Strategy

Recursive Merge Sort is a divide and conquer algorithm.

Divide and conquer means:

```
Divide the big problem into smaller problems.  
Solve the smaller problems.  
Combine the results.
```

For Recursive Merge Sort:

Divide

Split the array into two halves.

```
[8 3 7 4]
```

becomes:

```
[8 3]  [7 4]
```

Conquer

Sort the left half and the right half recursively.


```
[8 3] -> [3 8]  
[7 4] -> [4 7]
```

Combine

Merge the two sorted halves.

```
[3 8] and [4 7] -> [3 4 7 8]
```

7. Top-Down Recursion

Recursive Merge Sort is called a top-down method.

Why?

Because it starts with the whole array at the top.

Then it splits the array into smaller and smaller parts.

Example:

```
[8 3 7 4]
```

Split into:

```
[8 3]   [7 4]
```

Split again:

```
[8] [3] [7] [4]
```

Then merging begins.

So Recursive Merge Sort works from:

```
Whole array -> smaller halves -> single elements -> merged sorted array
```

8. Recursive Merge Sort vs Iterative Merge Sort

Both versions use the same `Merge()` operation.

The difference is how they create sorted groups.

Feature	Iterative Merge Sort	Recursive Merge Sort
Main control	Loops	Recursion
Direction	Bottom-up	Top-down
Starting idea	Treat every single element as sorted	Start with the whole array
Group creation	Merge size 1 into 2, 2 into 4, 4 into 8	Split into halves until one-element parts
Uses <code>Merge()</code> ?	Yes	Yes

Simple comparison:

```
Iterative Merge Sort = bottom-up using loops
Recursive Merge Sort = top-down using recursion
```

9. Main Idea of Recursive Merge Sort

Recursive Merge Sort works like this:

1. If the section has one element, stop.
2. Find the middle of the section.
3. Sort the left half using Merge Sort.
4. Sort the right half using Merge Sort.
5. Merge the two sorted halves.

Short memory trick:

```
Divide, sort left, sort right, merge.
```

10. The Four Logical Steps

Recursive Merge Sort has four main steps:

1. Divide
2. Sort left half
3. Sort right half
4. Merge both sorted halves

In code form:

```
mid = (low + high) / 2;

MergeSort(A, low, mid);
MergeSort(A, mid + 1, high);

Merge(A, low, mid, high);
```

The `Merge()` call comes last.

That is very important.

11. Meaning of `low`, `mid`, and `high`

Recursive Merge Sort uses three important positions:

Variable	Meaning
<code>low</code>	Starting index of the current section
<code>mid</code>	Middle index of the current section
<code>high</code>	Ending index of the current section

Example array:

```
Index:  0  1  2  3
Array:  8  3  7  4
```

At the beginning:

```
low = 0
high = 3
```

Now calculate `mid`:

```
mid = (low + high) / 2
mid = (0 + 3) / 2
mid = 1
```

So the array is divided like this:

```
Left half: A[0] to A[1] = 8 3
Right half: A[2] to A[3] = 7 4
```

Visually:

Index:	0	1	2	3
Array:	8	3	7	4
	low	mid	mid+1	high

12. Why We Use `mid = (low + high) / 2`

The line:

```
mid = (low + high) / 2;
```

finds the middle index of the current section.

Example:

```
low = 0
high = 7
```

Then:

```
mid = (0 + 7) / 2 = 3
```

So the section is split like this:

```
Left half: 0 to 3  
Right half: 4 to 7
```

The left half is:

```
A[0] to A[3]
```

The right half is:

```
A[4] to A[7]
```

13. The Base Case

A base case is the condition that stops recursion.

Without a base case, a recursive function would call itself forever.

In Merge Sort, the base case is handled by this condition:

```
if (low < high)
```

This means:

```
Only continue if the current section has at least two elements.
```

If:

```
low == high
```

then the section has one element.

Example:

```
[8]
```

One element is already sorted, so no more work is needed.

If the section has zero or one element, recursion stops.

14. Why the Merge Call Comes Last

This is one of the most important ideas in Phase 8.

The correct order is:

```
MergeSort(A, low, mid);  
MergeSort(A, mid + 1, high);  
Merge(A, low, mid, high);
```

This means:

```
First sort the left half.  
Then sort the right half.  
Then merge both sorted halves.
```

Why must `Merge()` come last?

Because merging works correctly only when both parts are already sorted.

Wrong idea:

```
Merge first, then sort left and right.
```

That is wrong because the two halves are not sorted yet.

Correct idea:

```
Sort left, sort right, then merge.
```

15. Post-Order Merging

Recursive Merge Sort uses post-order behavior.

Post-order means:

Process the smaller child parts first.
Then process the parent part.

In Merge Sort:

1. Split into child parts.
2. Sort child parts.
3. Merge them into the parent part.

At the bottom of the recursion tree, one-element lists are already sorted.

Then merging starts from the smallest pieces and builds upward.

16. Recursion Tree Idea

A recursion tree helps us see how the function calls split.

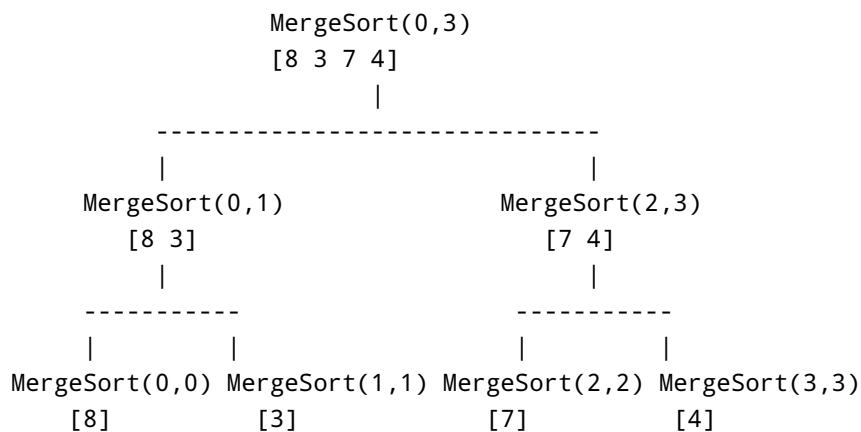
For this array:

[8 3 7 4]

The first call is:

MergeSort(0, 3)

The recursion tree looks like this:



The function calls go downward by splitting.

Then merging happens upward:

```
[8] + [3] -> [3 8]
[7] + [4] -> [4 7]
[3 8] + [4 7] -> [3 4 7 8]
```

17. Full Dry Run on [8, 3, 7, 4]

Let us sort this array:

```
8 3 7 4
```

The final sorted result should be:

```
3 4 7 8
```

Step 1: First Call

We call:

```
MergeSort(A, 0, 3);
```

Current section:

```
A[0] to A[3] = 8 3 7 4
```

Calculate mid:

```
mid = (0 + 3) / 2 = 1
```

Split into:

Left half: $A[0]$ to $A[1]$ = 8 3
Right half: $A[2]$ to $A[3]$ = 7 4

Step 2: Sort the Left Half [8, 3]

Call:

```
MergeSort(A, 0, 1);
```

Current section:

8 3

Calculate mid:

```
mid = (0 + 1) / 2 = 0
```

Split into:

[8] [3]

Call:

```
MergeSort(A, 0, 0);
```

This is one element:

[8]

So it stops.

Call:

```
MergeSort(A, 1, 1);
```

This is one element:

[3]

So it stops.

Now merge:

```
Merge(A, 0, 0, 1);
```

Merge:

[8] and [3]

Compare 8 and 3.

3 is smaller, so take 3 first.

Then copy leftover 8.

Result:

[3 8]

Now the left half is sorted.

Step 3: Sort the Right Half [7, 4]

Call:

```
MergeSort(A, 2, 3);
```

Current section:

7 4

Calculate mid:

```
mid = (2 + 3) / 2 = 2
```

Split into:

```
[7]  [4]
```

Call:

```
MergeSort(A, 2, 2);
```

This is one element:

```
[7]
```

So it stops.

Call:

```
MergeSort(A, 3, 3);
```

This is one element:

```
[4]
```

So it stops.

Now merge:

```
Merge(A, 2, 2, 3);
```

Merge:

```
[7] and [4]
```

Compare **7** and **4**.

4 is smaller, so take **4** first.

Then copy leftover 7.

Result:

[4 7]

Now the right half is sorted.

Step 4: Merge the Two Sorted Halves

Now we have:

[3 8] [4 7]

Call:

```
Merge(A, 0, 1, 3);
```

Now merge:

Left: [3 8]
Right: [4 7]

Compare 3 and 4.

3 is smaller, so copy 3.

Result so far: [3]

Compare 8 and 4.

4 is smaller, so copy 4.

Result so far: [3 4]

Compare 8 and 7.

7 is smaller, so copy 7.

Result so far: [3 4 7]

The right side is finished.

Copy leftover 8 from the left side.

Final result:

[3 4 7 8]

18. Dry Run Summary

Original array:

8 3 7 4

Split:

[8 3] [7 4]

Split again:

[8] [3] [7] [4]

Merge first pair:

[8] + [3] -> [3 8]

Merge second pair:

[7] + [4] -> [4 7]

Merge final two sorted halves:

[3 8] + [4 7] -> [3 4 7 8]

Final sorted array:

```
3 4 7 8
```

19. Dry Run on the Program Array

The program uses this array:

```
8 3 7 4 9 2 6 5
```

Recursive Merge Sort first splits it like this:

```
[8 3 7 4] [9 2 6 5]
```

Then:

```
[8 3] [7 4] [9 2] [6 5]
```

Then:

```
[8] [3] [7] [4] [9] [2] [6] [5]
```

Now merging begins:

```
[8] + [3] -> [3 8]  
[7] + [4] -> [4 7]  
[9] + [2] -> [2 9]  
[6] + [5] -> [5 6]
```

Then bigger merges:

```
[3 8] + [4 7] -> [3 4 7 8]  
[2 9] + [5 6] -> [2 5 6 9]
```

Final merge:

```
[3 4 7 8] + [2 5 6 9] -> [2 3 4 5 6 7 8 9]
```

Final sorted array:

```
2 3 4 5 6 7 8 9
```

20. Pseudocode for Recursive Merge Sort

Simple pseudocode:

```
MergeSort(A, low, high):  
    if low < high:  
        mid = (low + high) / 2  
        MergeSort(A, low, mid)  
        MergeSort(A, mid + 1, high)  
        Merge(A, low, mid, high)
```

Meaning:

```
If the section has more than one element:  
    Find the middle.  
    Sort the left half.  
    Sort the right half.  
    Merge both sorted halves.
```

21. The Merge() Function Recap

The Merge() function combines two sorted sections inside the same array.

Example:

```
A = [2, 6, 10, 3, 7, 12]
```

This array has two sorted sections:

```
Left section: [2, 6, 10]
Right section: [3, 7, 12]
```

After merging:

```
A = [2, 3, 6, 7, 10, 12]
```

22. Variables Used in Merge()

Variable	Meaning
A[]	Original array
low	Start of the left section
mid	End of the left section
mid + 1	Start of the right section
high	End of the right section
i	Moves through the left section
j	Moves through the right section
k	Writes into the temporary array B[]
B[]	Temporary array used for merging

23. Why We Need Temporary Array B[]

When merging two sections inside one array, we should not directly overwrite A[] too early.

Why?

Because we may destroy values before we use them.

So we use a temporary array:

```
int B[100];
```

The process is:

1. Compare values from left and right sections.
2. Copy the smaller value into B[].
3. Copy leftover values into B[].
4. Copy B[] back into A[].

Simple rule:

Merge safely into B[], then copy back to A[].

24. Complete Recursive Merge Sort Program in C

This program uses the stable merge condition:

```
if (A[i] <= A[j])
```

Why use `<=` ?

Because if two values are equal, taking the value from the left side first keeps duplicate values in their original order.

```
#include <stdio.h>

void Merge(int A[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int B[100];

    while (i <= mid && j <= high) {
        if (A[i] <= A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= mid) {
        B[k++] = A[i++];
    }

    while (j <= high) {
```

```

        B[k++] = A[j++];
    }

    for (i = low; i <= high; i++) {
        A[i] = B[i];
    }
}

void MergeSort(int A[], int low, int high) {
    int mid;

    if (low < high) {
        mid = (low + high) / 2;

        MergeSort(A, low, mid);
        MergeSort(A, mid + 1, high);

        Merge(A, low, mid, high);
    }
}

int main() {
    int A[] = {8, 3, 7, 4, 9, 2, 6, 5};
    int n = sizeof(A) / sizeof(A[0]);

    MergeSort(A, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}

```

25. Expected Output

```
Sorted array: 2 3 4 5 6 7 8 9
```

26. Explanation of the Complete Program

The program has three main parts:

1. Merge() function
2. MergeSort() function
3. main() function

27. Explanation of Merge() Function

Function header:

```
void Merge(int A[], int low, int mid, int high)
```

This function merges two sorted sections of the same array.

The left section is:

```
A[low] to A[mid]
```

The right section is:

```
A[mid + 1] to A[high]
```

Initialize `i`, `j`, and `k`

```
int i = low;  
int j = mid + 1;  
int k = low;
```

Meaning:

```
i starts at the beginning of the left section.  
j starts at the beginning of the right section.  
k starts at the position where we write into B[].
```

Temporary Array

```
int B[100];
```

This stores the merged result temporarily.

Beginner note:

`B[100]` means the code is safe only when the needed index range fits inside 100 positions.

For serious programs, the temporary array should be created based on the actual size of the input.

Main Merge Loop

```
while (i <= mid && j <= high)
```

This loop continues while both sides still have values.

It means:

```
The left section still has values.  
The right section still has values.
```

Both must be true.

Compare and Copy

```
if (A[i] <= A[j]) {  
    B[k++] = A[i++];  
} else {  
    B[k++] = A[j++];  
}
```

This compares the current value from the left section and the current value from the right section.

If the left value is smaller or equal, copy the left value.

Otherwise, copy the right value.

Meaning of `B[k++] = A[i++]`

This line:

```
B[k++] = A[i++];
```

means:

```
Copy A[i] into B[k].  
Move k to the next position.  
Move i to the next position.
```

It is a short form of:

```
B[k] = A[i];  
k++;  
i++;
```

Copy Leftover Values from Left Section

```
while (i <= mid) {  
    B[k++] = A[i++];  
}
```

If the right section finishes first, some values may still remain in the left section.

This loop copies them.

Copy Leftover Values from Right Section

```
while (j <= high) {  
    B[k++] = A[j++];  
}
```

If the left section finishes first, some values may still remain in the right section.

This loop copies them.

Copy `B[]` Back to `A[]`

```
for (i = low; i <= high; i++) {  
    A[i] = B[i];  
}
```

This step is very important.

The sorted merged result is inside `B[]`.

But the original array is `A[]`.

So we must copy the values back into `A[]`.

Without this step, the original array will not be updated correctly.

28. Explanation of `MergeSort()` Function

Function header:

```
void MergeSort(int A[], int low, int high)
```

This function sorts the array section from `low` to `high`.

Parameter meanings:

Parameter	Meaning
<code>A[]</code>	The array to sort
<code>low</code>	Starting index of the current section
<code>high</code>	Ending index of the current section

Declare `mid`

```
int mid;
```

`mid` stores the middle index of the current section.

Base Condition

```
if (low < high)
```

This means:

```
Only divide and sort if the section has more than one element.
```

If the section has one element, the function does nothing.

That is correct because one element is already sorted.

Find the Middle

```
mid = (low + high) / 2;
```

This divides the section into two halves.

Example:

```
low = 0  
high = 7  
mid = 3
```

So:

```
Left half: 0 to 3  
Right half: 4 to 7
```

Sort the Left Half

```
MergeSort(A, low, mid);
```

This recursively sorts the left half.

Sort the Right Half

```
MergeSort(A, mid + 1, high);
```

This recursively sorts the right half.

Merge the Sorted Halves

```
Merge(A, low, mid, high);
```

After the left and right halves are sorted, this line merges them into one sorted section.

29. Explanation of `main()` Function

```
int A[] = {8, 3, 7, 4, 9, 2, 6, 5};
```

This creates the array.

```
int n = sizeof(A) / sizeof(A[0]);
```

This finds the number of elements in the array.

Here:

```
n = 8
```

```
MergeSort(A, 0, n - 1);
```

This calls Merge Sort on the whole array.

Why `n - 1`?

Because array indexing starts from `0`.

If there are 8 elements, valid indexes are:


```
0 1 2 3 4 5 6 7
```

So the last valid index is:

```
n - 1 = 7
```

The loop:

```
for (int i = 0; i < n; i++) {  
    printf("%d ", A[i]);  
}
```

prints the sorted array.

30. Important Difference from Quick Sort Code

In the Quick Sort version from earlier phases, the code used an infinity marker:

```
∞
```

In C, that was represented as:

```
INT32_MAX
```

But Recursive Merge Sort does not use an infinity marker.

For Merge Sort, use real array indexes only.

So for an array of size `n`, call:

```
MergeSort(A, 0, n - 1);
```

Do not call:

```
MergeSort(A, 0, n);
```

Why?

Because `n` is outside the valid array index range.

31. Time Complexity of Recursive Merge Sort

Recursive Merge Sort has the same time complexity in best, average, and worst cases.

Case	Time complexity
Best case	<code>O(n log n)</code>
Average case	<code>O(n log n)</code>
Worst case	<code>O(n log n)</code>

32. Why Time Complexity Is `O(n log n)`

Merge Sort divides the array into halves.

Example:

```
n -> n/2 -> n/4 -> n/8 -> ... -> 1
```

The number of times we can divide by 2 is about:

```
log n
```

So the recursion tree has about `log n` levels.

At each level, all the merging work together processes about `n` elements.

Example:

```
Level 1: merge total n elements  
Level 2: merge total n elements  
Level 3: merge total n elements  
...
```

So total work is:

$n \text{ work per level} \times \log n \text{ levels}$

Therefore:

Time complexity = $O(n \log n)$

33. Why Best, Average, and Worst Cases Are the Same

Merge Sort divides the array in the same way no matter what the input looks like.

If the array is already sorted, it still divides and merges.

If the array is reverse sorted, it still divides and merges.

If the array is random, it still divides and merges.

So the time complexity stays:

$O(n \log n)$

This makes Merge Sort very consistent.

34. Space Complexity

Recursive Merge Sort needs extra memory.

It uses:

1. Auxiliary array B[]
2. Recursion call stack

The auxiliary array needs about:

$O(n)$

The recursion stack needs about:

```
0(log n)
```

Because the recursion depth follows repeated halving.

Overall:

```
0(n) + 0(log n)
```

Since `0(n)` is bigger than `0(log n)`, we say:

```
Overall extra space = 0(n)
```

35. Is Recursive Merge Sort In-Place?

No, not in this implementation.

An in-place algorithm sorts mostly inside the original array without needing a large extra array.

Recursive Merge Sort uses:

```
int B[100];
```

That means it needs extra memory.

So this implementation is not in-place.

Space complexity is:

```
0(n)
```

36. Is Recursive Merge Sort Stable?

Recursive Merge Sort can be stable.

Stable means:

Equal values keep their original relative order.

Example:

5A 3 5B

Here, 5A and 5B have the same value, but 5A came first originally.

After stable sorting:

3 5A 5B

5A still comes before 5B.

37. How to Keep Merge Sort Stable

During merging, when two values are equal, choose the value from the left side first.

Use:

```
if (A[i] <= A[j])
```

not only:

```
if (A[i] < A[j])
```

Why?

If the values are equal, <= copies the left-side value first.

That preserves the original order of duplicates.

Important note:

Using < will still sort numbers correctly, but it may not preserve the original order of equal records.

So for stability, prefer:

```
A[i] <= A[j]
```

38. Is Recursive Merge Sort Adaptive?

Recursive Merge Sort is usually not considered adaptive in this basic form.

Why?

Because it does almost the same splitting and merging whether the array is already sorted or not.

Example:

```
2 3 4 5 6 7 8 9
```

This array is already sorted.

But Merge Sort still:

```
splits it,  
sorts the halves recursively,  
merges them back.
```

So it does not become $O(n)$ just because the data is already sorted.

Its time stays:

```
 $O(n \log n)$ 
```

39. Is Recursive Merge Sort Comparison-Based?

Yes.

Merge Sort compares values during merging.

Example:

```
if (A[i] <= A[j])
```

This asks:

Is the left value smaller than or equal to the right value?

So Merge Sort is a comparison-based sorting algorithm.

40. Recursive Merge Sort Properties Table

Property	Recursive Merge Sort
Sorting type	Comparison-based
Main idea	Divide into halves, then merge sorted halves
Strategy	Divide and conquer
Direction	Top-down
Uses recursion?	Yes
Uses merging?	Yes
Best-case time	$O(n \log n)$
Average-case time	$O(n \log n)$
Worst-case time	$O(n \log n)$
Extra memory	$O(n)$
Recursion stack	$O(\log n)$
In-place?	No, not in this implementation
Stable?	Yes, if equal values are taken from the left side first
Adaptive?	Not in the basic version
Good for large data?	Yes, because time is consistent

41. Recursive Merge Sort vs Quick Sort

Both Merge Sort and Quick Sort use divide and conquer.

But they divide and combine differently.

Feature	Quick Sort	Recursive Merge Sort
Main operation	Partitioning	Merging
Key value	Pivot	Middle index
Divide method	Around pivot	Into two halves
Combine step	No special merge step	Must merge sorted halves
Best time	$O(n \log n)$	$O(n \log n)$
Average time	$O(n \log n)$	$O(n \log n)$
Worst time	$O(n^2)$	$O(n \log n)$
Extra array	Usually no large extra array	Needs auxiliary array
Stability	Usually not stable	Stable if merged correctly

Quick Sort memory trick:

Partition, sort left, sort right.

Merge Sort memory trick:

Split, sort left, sort right, merge.

42. Common Beginner Mistakes in Phase 8

Mistake 1: Calling `Merge()` Before Recursive Calls

Wrong order:

```
Merge(A, low, mid, high);
MergeSort(A, low, mid);
MergeSort(A, mid + 1, high);
```

Why wrong?

The left and right halves are not sorted yet.

Correct order:


```
MergeSort(A, low, mid);  
MergeSort(A, mid + 1, high);  
Merge(A, low, mid, high);
```

Mistake 2: Using `high = n` Instead of `n - 1`

Wrong:

```
MergeSort(A, 0, n);
```

Correct:

```
MergeSort(A, 0, n - 1);
```

Why?

If the array has `n` elements, the last valid index is `n - 1`.

Mistake 3: Forgetting the Base Case

Wrong idea:

```
Keep calling MergeSort forever.
```

Correct idea:

```
if (low < high)
```

This stops recursion when the section has one element or no elements.

Mistake 4: Thinking Merge Sort Uses the Infinity Marker

Quick Sort used an infinity marker in the earlier code.

Merge Sort does not use it.

Do not add `INT32_MAX` to the Merge Sort array.

Use real indexes only.

Mistake 5: Forgetting That Merging Needs Sorted Parts

Wrong idea:

Merge can sort any two random chunks.

Correct idea:

Merge works correctly only when both sections are already sorted.

Recursive Merge Sort makes the parts sorted first by splitting down to one-element parts and then merging upward.

Mistake 6: Forgetting to Copy Leftovers

Wrong idea:

```
while (i <= mid && j <= high) {  
    // compare and copy  
}
```

Then stopping.

Problem:

One side may still have values left.

Correct code:

```
while (i <= mid) {  
    B[k++] = A[i++];  
}  
  
while (j <= high) {  
    B[k++] = A[j++];  
}
```

Mistake 7: Forgetting to Copy `B[]` Back to `A[]`

After merging, the sorted values are inside `B[]`.

If we forget this:

```
for (i = low; i <= high; i++) {  
    A[i] = B[i];  
}
```

then the original array `A[]` will not be updated.

Mistake 8: Using a Very Small Fixed Temporary Array

This code uses:

```
int B[100];
```

This is okay for small learning examples.

But if the array is larger than 100 positions, this can fail.

For serious programs, create the temporary array based on the input size.

Mistake 9: Thinking Merge Sort Is `O(1)` Space

Merge Sort is fast, but it uses extra memory.

This implementation uses an auxiliary array.

So extra space is:

```
O(n)
```

not:

```
O(1)
```

Mistake 10: Thinking Merge Sort Becomes $O(n)$ on Sorted Input

Basic Merge Sort does not stop early like adaptive Bubble Sort.

Even if the array is already sorted, it still divides and merges.

So time remains:

```
O(n log n)
```

Mistake 11: Using $<$ When You Need Stability

This condition:

```
if (A[i] < A[j])
```

sorts numbers correctly.

But for duplicate records, it may break stability.

For stable Merge Sort, use:

```
if (A[i] <= A[j])
```

Mistake 12: Confusing Recursive Merge Sort with Iterative Merge Sort

Iterative Merge Sort uses loops.

Recursive Merge Sort uses function calls.

Simple difference:

```
Iterative = loops  
Recursive = function calls itself
```

43. Phase 8 Key Takeaways

- Recursive Merge Sort is a divide and conquer sorting algorithm.
- It starts with the whole array.
- It splits the array into two halves.
- It keeps splitting until each part has one element.
- One element is already sorted.
- It merges sorted parts while returning from recursion.
- The merge step must happen after the recursive calls.
- This is called post-order merging.
- `low` is the start of the current section.
- `high` is the end of the current section.
- `mid` is the middle point.
- The left half is `low` to `mid`.
- The right half is `mid + 1` to `high`.
- The base condition is `if (low < high)`.
- The `Merge()` function uses `i`, `j`, `k`, and `B[]`.
- `i` reads the left section.
- `j` reads the right section.
- `k` writes into the temporary array.
- `B[]` stores the merged result temporarily.
- Leftover elements must be copied.
- `B[]` must be copied back to `A[]`.
- Best-case time is $O(n \log n)$.
- Average-case time is $O(n \log n)$.
- Worst-case time is $O(n \log n)$.
- Extra space is $O(n)$.
- Recursion stack space is $O(\log n)$.
- Overall extra space is $O(n)$.
- Merge Sort is not in-place in this implementation.
- Merge Sort can be stable if equal values are taken from the left side first.
- Recursive Merge Sort uses recursion, unlike Iterative Merge Sort.

44. Final Beginner Memory Trick

Remember Phase 8 like this:

Split the array into halves.
Keep splitting until one-element parts.
Merge sorted parts back together.

Even shorter:

Split down, merge up.

Or:

Divide first, merge last.

The most important sentence of Phase 8 is:

Recursive Merge Sort uses recursion to split the array, then uses merging to build the sorted array back up.